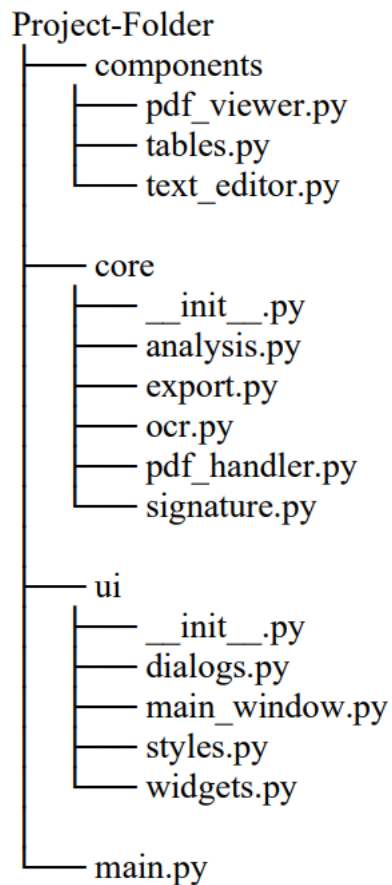


CODE EXPLANATION



1. Components

1.1.pdf_viewer.py

Library	Purpose
fitz (PyMuPDF)	PDF page handling and text extraction from selected regions
PyQt6.QtWidgets	GUI components such as labels, menus, rubber-band selection, application access
PyQt6.QtCore	Signals, timers, geometry handling, mouse interaction support
PyQt6.QtGui	Image/pixmap handling for extracted PDF regions
PyQt6.QtWebEngineWidgets	Embedded PDF viewing using Chromium-based web engine
PyQt6.QtWebEngineCore	Web page actions (copy, reload, settings) inside PDF viewer

Classes used:

1. PDFWebEngineView

Method	Purpose
contextMenuEvent()	Creates a custom right-click menu. Adds copy, paste, summarize, and reload options. Connects menu actions to custom handlers.
copy_and_paste()	Triggers WebEngine copy action. Waits briefly for clipboard update. Emits copied text through signal.
copy_and_summarize()	Copies selected text. Reads clipboard content. Emits summary request signal.

2. PdfPageLabel

Method	Purpose
__init__()	Stores page reference. Creates selection rectangle. Initializes panning state variables.
mousePressEvent()	Left click starts selection rectangle. Right/Middle click enables panning mode.
mouseMoveEvent()	Updates rubber-band selection size. Moves scrollbars during panning.
mouseReleaseEvent()	Finalizes selection rectangle. Opens extraction menu. Resets panning state.
get_text_from_rect()	Converts screen coordinates to PDF coordinates. Creates PDF rectangle. Uses PyMuPDF textbox extraction.
show_context_menu()	Displays extraction/OCR actions(right click options)

1.2.tables.py

Classes:

ProjectTablesDialog

Function	Purpose
<code>__init__(self, parent=None)</code>	Initializes the Project Tables dialog, creates all tabs, layouts, buttons, and loads existing data from the parent application.
<code>_create_issue_details_tab(self)</code>	Creates the "Issue Details" tab containing File No., Issue No., and Date of Issue fields.
<code>_set_issue_detail(self, key, value)</code>	Updates the corresponding issue detail value in the parent application's data model whenever a field changes.
<code>_create_stakeholders_tab(self)</code>	Creates the Stakeholders tab with input fields, add button, and editable stakeholders table.
<code>_create_cert_task_tab(self)</code>	Creates the Certification Task Allocation tab with inputs and editable task allocation table.
<code>_create_generic_table_tab(self, tab_name, data_attr, title)</code>	Dynamically creates configurable tabs such as Annexure-1, Test Rigs, Aircraft Checks, and Annexure-2 based on predefined column configurations.
<code>_configure_table(self, table)</code>	Applies common table settings such as row selection, alternating row colors, and column stretching.
<code>_row_action_layout(self, delete_handler, clear_handler)</code>	Creates a reusable button layout containing "Delete Selected Row" and "Clear All" actions.
<code>refresh_all_tables(self)</code>	Reloads all tabs and tables from the parent application's current data.
<code>add_stakeholder(self)</code>	Adds a new stakeholder entry to the stakeholders dataset and refreshes the table.

Function	Purpose
on_sh_cell_changed(self, row, column)	Updates stakeholder data when a user edits a stakeholder table cell.
delete_selected_sh(self)	Deletes the currently selected stakeholder row.
clear_all_sh(self)	Removes all stakeholder entries from the dataset.
refresh_sh_table(self)	Rebuilds and refreshes the stakeholder table from stored data.
add_task(self)	Adds a new certification task allocation record.
on_ct_cell_changed(self, row, column)	Updates certification task data when a table cell is modified.
delete_selected_ct(self)	Deletes the selected certification task row.
clear_all_ct(self)	Removes all certification task entries.
refresh_ct_table(self)	Refreshes the certification task table using current data.
add_generic_row(self, data_attr)	Adds a new row to one of the dynamically generated tables (Annexure-1, Test Rigs, etc.).
on_generic_cell_changed(self, data_attr, row, column)	Updates underlying data when a user edits a cell in a generic table.
delete_selected_generic(self, data_attr)	Deletes the selected row from a specified generic table.
clear_all_generic(self, data_attr)	Clears all records from a specified generic table.
refresh_generic_table(self, data_attr)	Rebuilds and refreshes a generic table from stored data.
_set_sl_item(self, table, row)	Creates and inserts a non-editable serial number (SI No.) into a table row.

1.3.text_editor.py

Classes:

Class	Purpose
NotesEditor	Custom rich text editor that supports pasting both images and text from clipboard/mime data.
SelectableTextPreview	Read-only text preview widget that allows text selection and provides extraction/summarization actions through a context menu.

Function	Purpose
canInsertFromMimeData(self, source)	Checks whether incoming clipboard/mime data contains text or images that can be inserted into the editor.
insertFromMimeData(self, source)	Handles custom paste operations. Inserts images (with resizing) or plain text into the notes editor.
__init__(self, parent=None)	Initializes the read-only preview widget and applies custom styling.
contextMenuEvent(self, event)	Creates a custom right-click menu that allows text extraction and summarization of selected text.

2. Core

2.1.analysis.py

Classes:

Class	Purpose
AnalysisWorker	Analyzes extracted document text, detects predefined topics, extracts project name and introduction, and generates summaries using AI or offline algorithms.
FileDetailsWorker	Extracts metadata from PDF documents using OCR and native PDF text, validates signatures, and generates structured file details.

Class	Purpose
SignatureValidationWorker	Performs standalone signature validation on a PDF document in a background thread.

Method	Purpose
<code>__init__(text, topics)</code>	Initializes the analysis worker with document text and checklist topics.
<code>clean_extracted_name(name)</code>	Cleans and normalizes extracted project names.
<code>run()</code>	Main execution method that analyzes document content and emits results.
<code>_is_heading_line(line)</code>	Determines whether a line resembles a section heading.
<code>_heading_found_in_text(keyword_set)</code>	Checks if keywords appear together in a heading line.
<code>_keywords_found_in_window(keywords, text, window=120)</code>	Checks if keywords appear close together within a text window.
<code>_topic_found_in_text(topic, text)</code>	Determines whether a specific topic exists in the document.
<code>_extract_introduction()</code>	Extracts the Introduction section from the document.
<code>_try_ai_summarization(intro_text, is_short)</code>	Uses a local AI model to summarize content or extract project information.
<code>__init__(pdf_handler)</code>	Initializes the worker with a PDF handler.
<code>run()</code>	Performs OCR, extracts text, validates signatures, and generates file details.
<code>preprocess_for_ocr(image)</code>	Applies image enhancement and preprocessing before OCR.
<code>detect_signature(img_array)</code>	Detects signatures within an image region.
<code>extract_details(text)</code>	Extracts structured metadata from document text.
<code>__init__(pdf_handler)</code>	Initializes the signature validation worker.
<code>run()</code>	Validates whether the PDF contains a valid signature and emits the result.

2.2.export.py

This file mainly generates final template document.

2.3.ocr.py

functions:

Function	Purpose
<code>pixmap_to_rgb_array(pix)</code>	Converts a PyMuPDF pixmap into a NumPy RGB image array.
<code>rect_to_pdf_rect(page, view_width, view_height, rect, padding_points=2.0)</code>	Converts screen/view coordinates into PDF coordinates with optional padding.
<code>render_pdf_region(page, pdf_rect, dpi=500, max_pixels=24000000)</code>	Renders a selected PDF region as a high-resolution RGB image.
<code>_deskew(gray)</code>	Detects and corrects image skew/rotation before OCR.
<code>build_ocr_variants(image)</code>	Generates multiple preprocessed image variants to improve OCR accuracy.
<code>_clean_text(text)</code>	Cleans OCR output by removing unwanted characters and formatting issues.
<code>_normalize_details_text(text)</code>	Normalizes extracted text by replacing special characters and standardizing whitespace.
<code>_compact_details_text(text)</code>	Compresses and cleans text for easier pattern matching.
<code>_clean_field_value(value)</code>	Cleans extracted field values by removing punctuation and extra spaces.
<code>_first_match(text, patterns)</code>	Returns the first successful regex match from a list of patterns.
<code>_looks_like_project_label(line)</code>	Determines whether a line is a metadata label rather than a project name.
<code>_is_project_candidate(line)</code>	Validates whether a line is likely to be a project name candidate.
<code>_extract_project_name_from_lines(text)</code>	Attempts to extract the project name from document lines and headings.
<code>extract_file_details_from_text(text)</code>	Extracts File No, Issue No, Date of Issue, Project Name, and signature status from document text.
<code>_confidence_score(data)</code>	Calculates average OCR confidence from Tesseract output data.
<code>_text_quality_score(text, confidence)</code>	Computes a quality score for OCR text using readability and confidence metrics.
<code>run_tesseract_ocr(image, lang="eng")</code>	Runs OCR on multiple image variants/configurations and returns the best result.

Function	Purpose
extract_region_text(page, view_width, view_height, rect, lang="eng")	Extracts text from a selected PDF region using native PDF text and OCR fallback logic.

2.4.pdf_handler.py

Class	Purpose
PDFHandler	Manages PDF document operations including opening PDFs, extracting pages as images, loading specific pages, and closing documents.

Method	Purpose
__init__(self)	Initializes the PDF handler and prepares storage for document and page data.
open_pdf(self, path)	Opens a PDF file, loads the document, and extracts all text content from its pages.
extract_pages(self, max_pages=15)	Renders PDF pages as images (QPixmap objects) and stores them for display.
get_page(self, index)	Returns a specific PDF page object by index.
close(self)	Closes the currently loaded PDF document and releases resources.

2.5.signature.py

Function	Purpose
page_to_rgb_array(page, zoom=2.5)	Converts a PDF page into an RGB NumPy image array for image processing.
detect_signature_in_image(img_array)	Detects whether a signature-like pattern exists within an image using color and grayscale analysis.
_mask_has_signature(ink, min_height_ratio)	Evaluates contour characteristics to determine if detected ink regions resemble handwritten signatures.
document_has_valid_signature(pdf_doc)	Scans all pages in a PDF document and returns whether a valid signature is detected anywhere in the document.

3. UI

3.1.dialogs.py

classes:

Class	Purpose
PDFViewerDialog	Displays generated PDFs with zoom controls using either QWebEngineView or a fallback image-based PDF renderer.
FileDetailsDialog	Displays extracted file metadata, allows user review/editing, and saves the details back to the application.

Method	Class	Purpose
__init__(self, pdf_path, parent=None)	PDFViewerDialog	Initializes the PDF preview dialog and loads the PDF viewer.
zoom_in(self)	PDFViewerDialog	Increases PDF zoom level.
zoom_out(self)	PDFViewerDialog	Decreases PDF zoom level.
apply_zoom(self)	PDFViewerDialog	Applies the current zoom factor to the active PDF viewer.
_add_fallback_viewer(self, layout, pdf_path)	PDFViewerDialog	Creates a fallback PDF viewer using rendered page images when WebEngine is unavailable.
render_fallback_pages(self)	PDFViewerDialog	Renders PDF pages as images and updates the fallback viewer display.
__init__(self, parent=None)	FileDetailsDialog	Initializes the file details dialog, form fields, status display, and action buttons.
save_details(self)	FileDetailsDialog	Collects edited file details and emits them to the parent application.
set_extracting_state(self, is_extracting)	FileDetailsDialog	Updates UI state while OCR/file detail extraction is running.

Method	Class	Purpose
update_results(self, results)	FileDetailsDialog	Populates the dialog fields with extracted OCR/file metadata results.

3.2.main_window.py

Classe:

Class	Purpose
OfflineApp	Main application window that manages PDF viewing, OCR, document analysis, note-taking, checklist validation, table management, metadata extraction, and document export.

Method	Purpose
manual_input (<i>property</i>)	Returns the currently active notes editor based on the selected heading.
store_selected_region(pixmap, rect, page)	Stores a user-selected PDF region for later OCR processing.
perform_selected_ocr()	Initiates OCR on the currently selected PDF region.
perform_region_ocr(pixmap, rect, page)	Extracts text from a selected PDF region and displays the OCR result.
get_all_manual_notes_html()	Returns all manual notes in HTML format.
get_all_manual_notes_text()	Returns all manual notes as plain text.
__init__()	Initializes application state, data structures, dialogs, workers, and UI components.
init_ui()	Creates the main application layout and initializes left/right panels.
_create_top_toolbar()	Creates the top toolbar containing upload controls.
_create_left_panel()	Builds the left panel containing PDF viewing and OCR tools.
_setup_left_tabs()	Configures PDF viewer and text preview tabs.
_create_right_panel()	Builds the assistant workspace panel and export controls.
_create_status_group()	Creates the checklist/status tracking section.
save_checklist_as_image()	Saves the checklist panel as a PNG image.

Method	Purpose
<code>_create_params_section()</code>	Creates configurable project parameter selection controls.
<code>init_data_dialog()</code>	Creates the project data and notes management dialog.
<code>init_tables_dialog()</code>	Creates and initializes the project tables dialog.
<code>get_docx_table_data()</code>	Collects all table data for document generation.
<code>show_data_popup(tab_index=0)</code>	Displays the project data dialog.
<code>show_tables_popup()</code>	Opens the project tables management dialog.
<code>show_file_details()</code>	Opens the file details extraction dialog and starts metadata extraction.
<code>save_file_details_to_document(details)</code>	Saves extracted file metadata into application data structures.
<code>flash_data_icon()</code>	Visually indicates that data has been updated.
<code>paste_to_notes()</code>	Pastes clipboard text into the active notes editor.
<code>summarize_clipboard()</code>	Generates a summary from clipboard text.
<code>generate_summary_from_selection(text)</code>	Generates and stores a summary from selected text.
<code>add_extracted_text(text)</code>	Inserts extracted text into the active notes section.
<code>add_extracted_image(pixmap)</code>	Inserts an extracted image into the active notes section.
<code>set_annexure3_image(pixmap)</code>	Stores an image for inclusion in Annexure-3 during document generation.
<code>showEvent(event)</code>	Handles initial window display events and splitter sizing.
<code>set_initial_splitter_sizes()</code>	Sets the default 50/50 splitter layout.
<code>eventFilter(source, event)</code>	Handles Ctrl+MouseWheel zoom events.
<code>zoom_in(step=50)</code>	Increases PDF preview zoom level.
<code>zoom_out(step=50)</code>	Decreases PDF preview zoom level.
<code>refresh_pdf_view()</code>	Rebuilds and refreshes displayed PDF page previews.
<code>upload_file()</code>	Loads PDF/TXT files and initializes analysis workflows.
<code>start_analysis_thread(text)</code>	Starts background document analysis.
<code>start_signature_validation_thread()</code>	Starts background signature validation.
<code>handle_signature_validation_done(has_signature)</code>	Updates checklist based on signature validation results.

Method	Purpose
handle_signature_validation_error(message)	Handles signature validation errors.
handle_analysis_done(results)	Processes document analysis results and updates the UI.
export_pdf()	Generates and previews a PDF version of the document.
_write_docx(save_path)	Generates the final DOCX document.
export_docx()	Exports the completed document as a Word file.

3.3.styles.py

this file contains the centralized UI styling configuration for the PyQt application.

3.4.widgets.py

classes:

Class	Purpose
CollapsibleBox	Reusable collapsible/expandable container widget used to show or hide grouped UI content.
DropdownPopup	Popup dialog that provides searchable multi-select functionality with checkboxes, select-all, clear, and add-option features.
PopupMultiSelect	Custom dropdown button widget that displays selected items and launches the multi-select popup.

Method	Class	Purpose
__init__(self, title="", parent=None)	CollapsibleBox	Initializes the collapsible container and its content area.
on_press(self)	CollapsibleBox	Toggles between expanded and collapsed states.
__init__(self, parent_btn, options)	DropdownPopup	Creates the popup dialog with search, selection, and option management controls.
hideEvent(self, event)	DropdownPopup	Updates parent widget state when the popup closes.

Method	Class	Purpose
filter_options(self, text)	DropdownPopup	Filters visible options based on search text.
select_visible_options(self)	DropdownPopup	Selects all currently visible options.
clear_visible_options(self)	DropdownPopup	Unchecks all currently visible options.
on_item_pressed(self, item)	DropdownPopup	Handles checkbox toggling when list items are clicked.
add_item(self, text, checked=False)	DropdownPopup	Adds a new selectable item to the list.
show_new_input(self)	DropdownPopup	Displays the input field for adding custom options.
save_new_option(self)	DropdownPopup	Saves a newly entered custom option and adds it to the list.
__init__(self, options, parent=None)	PopupMultiSelect	Initializes the multi-select dropdown button and popup management system.
sizeHint(self)	PopupMultiSelect	Returns the preferred widget size.
resizeEvent(self, event)	PopupMultiSelect	Repositions the dropdown arrow button during resizing.
mousePressEvent(self, event)	PopupMultiSelect	Opens the popup when the widget is clicked.
toggle_popup(self)	PopupMultiSelect	Toggles popup visibility.
set_popup_open(self, is_open)	PopupMultiSelect	Updates popup open/closed state and arrow indicator.
show_popup(self)	PopupMultiSelect	Displays and positions the popup relative to screen boundaries.
update_display(self)	PopupMultiSelect	Updates button text to reflect selected items.
checkedItems(self)	PopupMultiSelect	Returns a list of all selected items.

4. main.py

This is the **application entry-point (startup) file** for the Document Extraction Assistant. Its sole responsibility is to initialize the application environment, configure the PyQt6 application settings, apply the UI theme, launch the main window, and start the event loop.

main()- Creates and configures the PyQt application, applies styling/theme settings, launches the main window, and starts the application event loop.